



Fuel Capacity Optimization System Documentation

Scope: Complete documentation of the convergent fuel capacity optimization system, including fixed-point iteration with Aitken's Δ^2 acceleration, dynamic mass evolution tracking, performance metrics calculation, and integration with 3D Dynamic Programming for climb and descent trajectory generation. The system determines minimum required fuel capacity through iterative convergence while maintaining mission feasibility and safety margins.

Table of Contents

1. [System Overview and Objectives](#)
2. [Mathematical Foundation](#)
3. [System Architecture and Data Structures](#)
4. [Convergence Algorithm Implementation](#)
5. [Aitken Acceleration Theory and Application](#)
6. [Mission Physics Integration](#)
7. [Performance Metrics Calculation](#)
8. [Code Execution Flow and Logic](#)
9. [Integration and Interface](#)
10. [Validation and Quality Assurance](#)
11. [Convergence Challenges and Analysis](#)
12. [Advanced Topics and Extensions](#)

1) System Overview and Objectives

1.1 Purpose and Scope

The fuel capacity optimization system implements a sophisticated convergent iterative

algorithm to determine the minimum required fuel capacity for mission completion. This approach replaces static, user-defined Maximum Take-Off Fuel (MTOF) values with dynamically optimized minimum fuel loads, eliminating superfluous fuel mass and improving aircraft performance through systematic optimization enhanced with Aitken's Δ^2 acceleration method.

1.2 System Objectives

Primary Objectives:

- **Fuel Minimization:** Determine minimum required fuel capacity for mission completion
- **Convergent Optimization:** Implement iterative refinement with Aitken acceleration until fuel load converges
- **Safety Integration:** Apply systematic safety buffers to optimized results
- **Performance Tracking:** Monitor comprehensive performance metrics throughout optimization
- **Mission Integration:** Coordinate climb, cruise, and descent phase optimization with dynamic mass evolution

Key Components:

- Convergent iterative optimization loop with adaptive damping
- Aitken's Δ^2 acceleration for quadratic convergence enhancement
- Dynamic fuel load adjustment mechanism with physical consistency checks
- Multi-phase mission simulation integration (climb + cruise + descent)
- Comprehensive performance parameter evolution tracking
- Safety buffer application system with configurable margins
- Error handling and recovery mechanisms for robust operation

1.3 System Flow Overview

The optimization system follows a systematic progression:

1. **Initialization:** Start with maximum fuel capacity as initial guess
2. **Mission Simulation:** Execute complete mission (climb + cruise + descent) with current fuel load
3. **Convergence Analysis:** Compare fuel consumed vs. initial fuel load using relative

tolerance

4. **Aitken Acceleration:** Compute adaptive damping factor from convergence history (iter ≥ 3)
 5. **Iteration Update:** Update fuel load using damped relaxation with adaptive parameter
 6. **Convergence Detection:** Monitor relative fuel change until within specified tolerance
 7. **Safety Application:** Apply safety buffer (5%) to converged result
 8. **Visualization:** Generate comprehensive convergence analysis plots
-

2) Mathematical Foundation

2.1 Optimization Problem Formulation

Objective: Determine minimum fuel capacity $F_{\min}$ that ensures mission completion:

$$\text{minimize: } F_{total} = F_{climb} + F_{cruise} + F_{descent}$$

Subject to constraints:

$$\text{Mission completion: All phases feasible with fuel } F_{total} \quad (1)$$

$$\text{Aircraft performance: Within flight envelope limits} \quad (2)$$

$$\text{Safety margin: } F_{capacity} = F_{total} \times (1 + \beta_{safety}) \quad (3)$$

Where:

- F_{total} : Total mission fuel consumption [kg]
- F_{climb} , F_{cruise} , $F_{descent}$: Phase-wise fuel consumption [kg]
- β_{safety} : Safety buffer percentage (typically 5%)

2.2 Fixed-Point Iteration Theory

Basic Fixed-Point Formulation:

The fuel optimization problem can be formulated as a fixed-point problem:

$$F = g(F)$$

Where:

- F : Initial fuel load [kg]
- $g(F)$: Mission simulation function returning fuel consumed [kg]
- Fixed point: F^* such that $F^* = g(F^*)$

Iterative Scheme:

$$F_{k+1} = g(F_k)$$

Damped Update for Stability:

$$F_{k+1} = \omega \cdot g(F_k) + (1 - \omega) \cdot F_k$$

Where:

- ω : Damping factor (relaxation parameter), $0 < \omega \leq 1$
- F_k : Initial fuel for iteration k [kg]
- $g(F_k)$: Fuel consumed in iteration k [kg]

Convergence Criterion:

$$\left| \frac{F_{k+1} - F_k}{F_k} \right| < \epsilon_{tol}$$

Where $\epsilon_{tol} = 0.005$ (0.5% relative tolerance)

2.3 Aitken's Δ^2 Acceleration Method

Theoretical Foundation:

Aitken's Δ^2 process (Aitken, 1926) accelerates linearly convergent fixed-point iterations to achieve quadratic convergence through adaptive relaxation parameter computation.

Mathematical Formulation:

For a sequence $\{F_k\}$ generated by fixed-point iteration:

$$\Delta F_k = g(F_k) - g(F_{k-1}) \quad (4)$$

$$\Delta F_{k-1} = g(F_{k-1}) - g(F_{k-2}) \quad (5)$$

$$\Delta^2 F_k = \Delta F_k - \Delta F_{k-1} \quad (6)$$

Aitken Acceleration Factor:

$$\alpha_k = 1 - \frac{\Delta F_k}{\Delta^2 F_k}$$

Adaptive Damping Update:

$$\omega_k = \omega_{k-1} \times \alpha_k$$

Bounded Damping:

$$\omega_k \in [\omega_{min}, \omega_{max}] = [0.1, 0.9]$$

Accelerated Update:

$$F_{k+1} = \omega_k \cdot g(F_k) + (1 - \omega_k) \cdot F_k$$

Convergence Rate:

- **Without Aitken** (fixed ω): Linear convergence, $O(\omega^n)$
- **With Aitken** (adaptive ω): Quadratic convergence near fixed point

2.4 Safety Buffer Application

Safety Margin Formulation:

$$F_{capacity} = F_{converged} \times (1 + \beta)$$

Where:

- $F_{\{converged\}}$: Converged fuel consumption [kg]
- $\beta = 0.05$: Safety buffer (5%)
- $F_{\{capacity\}}$: Final optimized fuel capacity [kg]

Purpose:

- Accounts for operational variabilities

- Provides contingency fuel reserve
 - Ensures mission completion under off-nominal conditions
-

3) System Architecture and Data Structures

3.1 System Parameters

Convergence Control Parameters:

```
class ConvergenceParameters:
    CONVERGENCE_TOLERANCE_RELATIVE = 0.005 # 0.5% relative tolerance
    CONVERGENCE_TOLERANCE_PERCENT = 0.5 # 0.5% in percentage units
    SAFETY_BUFFER_PERCENT = 0.05 # 5% safety buffer
    MAX_ITERATIONS = 5 # Safety limit
    DAMPING_FACTOR = 0.4 # Initial relaxation parameter
    USE_AITKEN_ACCELERATION = True # Enable Aitken's  $\Delta^2$  method
    AITKEN_MIN_DAMPING = 0.1 # Minimum damping factor
    AITKEN_MAX_DAMPING = 0.9 # Maximum damping factor
```

Mission Configuration Parameters:

```
TARGET_ALT_CLIMB_M = 10000.0 # Target cruise altitude [m]
START_ALTITUDE_CLIMB_M = 10.0 # Initial climb altitude [m]
CRUISE_DISTANCE_KM = 1500.0 # Cruise distance [km]
TARGET_DESCENT_ALT_M = 300.0 # Target descent altitude [m]
TARGET_DESCENT_MACH = 0.25 # Target approach Mach
```

3.2 Data Structures

Mission Iteration Results:

```

@dataclass
class MissionIterationResults:
    """
    Complete results from a single mission iteration.

    Required fields:
        iteration: Iteration number
        initial_fuel_kg: Initial fuel load [kg]
        initial_mass_kg: Initial total mass [kg]
        fuel_consumed_kg: Total fuel consumed [kg]
        convergence_delta_percent: Relative change from previous iteration [%]
        climb_result: Climb phase results (MinFuelSchedule)
        cruise_result: Cruise phase results (CruiseResults)
        descent_result: Descent phase results (DescentResults)
        total_time_s: Total mission duration [s]
        total_distance_km: Total ground distance [km]
        final_weight_kg: Final aircraft weight [kg]
        climb_fuel_kg, cruise_fuel_kg, descent_fuel_kg: Phase fuel [kg]
        climb_time_s, cruise_time_s, descent_time_s: Phase time [s]

    Optional performance metrics (with defaults):
        avg_lift_{phase}_N: Average lift force per phase [N]
        avg_drag_{phase}_N: Average drag force per phase [N]
        avg_ld_{phase}: Average L/D ratio per phase
        avg_lever_{phase}: Average thrust lever per phase
        avg_specific_energy_{phase}_J_kg: Average specific energy per phase [J/kg]
    """

```

Convergence History:

```
@dataclass
class ConvergenceHistory:
    """
    Tracking structure for convergence analysis.

    Attributes:
        iterations: List[MissionIterationResults]

    Methods:
        add_iteration(result): Add iteration to history
        get_last_two_iterations(): Retrieve last two iterations
        is_converged(): Check convergence criterion
    """
```

4) Convergence Algorithm Implementation

4.1 Optimization Loop Structure

Function: `FuelOptimizationCore.ConvergenceController.optimize_fuel_capacity()`

Purpose: Execute iterative optimization to determine minimum fuel capacity.

Algorithm Flow:


```

def optimize_fuel_capacity(aero, eng, M_grid, H_plot, lever_samples=50):
    """
    Main optimization loop with Aitken acceleration.

    Process:
    1. Initialize:  $F_0 = \text{MAX\_FUEL\_KG}$ 
    2. For  $k = 1, 2, \dots, \text{MAX\_ITERATIONS}$ :
        a. Run mission with  $F_k$ 
        b. Record fuel consumed:  $g(F_k)$ 
        c. Compute convergence delta
        d. Check convergence criterion
        e. If converged: Apply safety buffer and terminate
        f. Apply Aitken acceleration (if  $k \geq 3$ )
        g. Update:  $F_{k+1} = \omega_k \times g(F_k) + (1-\omega_k) \times F_k$ 
    3. Return optimized result with history
    """

    # Initialization
    initial_fuel_current_kg = MAX_FUEL_KG
    history = ConvergenceHistory()
    current_damping = DAMPING_FACTOR
    iteration_count = 0

    # Iterative loop
    while iteration_count < MAX_ITERATIONS:
        iteration_count += 1

        # Calculate total mass
        current_total_mass = W_OE + W_PL + initial_fuel_current_kg

        # Execute mission simulation
        iteration_result = run_single_mission_iteration(
            initial_mass_kg=current_total_mass,
            aero=aero, eng=eng, M_grid=M_grid, H_plot=H_plot,
            lever_samples=lever_samples, print_progress=True
        )

        # Process results and check convergence
        iteration_result.iteration = iteration_count

```

```

if iteration_count > 1:
    delta = compute_convergence_delta(iteration_result, history.iterations[-1])
    iteration_result.convergence_delta_percent = delta

history.add_iteration(iteration_result)

# Convergence check
if history.is_converged():
    optimized_fuel = iteration_result.fuel_consumed_kg * (1 + SAFETY_BUFFER)
    break

# Aitken acceleration
if USE_AITKEN and len(history.iterations) >= 3:
    current_damping = apply_aitken_acceleration(history, current_damping)

# Update fuel for next iteration
fuel_update = current_damping * iteration_result.fuel_consumed_kg + \
    (1 - current_damping) * initial_fuel_current_kg
initial_fuel_current_kg = fuel_update

return history.iterations[-1], history

```

4.2 Convergence Detection

Function: `ConvergenceHistory.is_converged()`

Purpose: Determine if optimization has converged based on relative fuel change.

Implementation:

```
def is_converged(self) -> bool:
    """
    Check convergence criterion.

    Mathematical condition:
        |Δf_rel| < ε_tolerance

    where:
        Δf_rel = (F_consumed,k - F_consumed,k-1) / F_consumed,k-1
        ε_tolerance = 0.5%
    """

    if len(self.iterations) < 2:
        return False

    prev, curr = self.get_last_two_iterations()
    delta = curr.convergence_delta_percent

    return abs(delta) < CONVERGENCE_TOLERANCE_PERCENT
```

Convergence Interpretation:

Delta	Status	Action
< 0.5%	Converged	Terminate, apply safety buffer
0.5% - 2%	Near convergence	Continue, monitor closely
2% - 10%	Converging	Continue iteration
> 10%	Slow convergence	Check damping factor

4.3 Mission Iteration Execution

Function: FuelOptimizationCore.IterationExecutor.run_single_mission_iteration()

Purpose: Execute complete mission simulation with specified initial mass.

Execution Sequence:

- 1. Climb Phase:

- Solve 3D DP optimization with current mass
- Calculate fuel consumed and time elapsed
- Update mass: $m_{\text{cruise}} = m_{\text{initial}} - F_{\text{climb}}$

2. **Cruise Phase:**

- Initialize from climb endpoint state
- Simulate steady-level cruise
- Calculate fuel consumed and time elapsed
- Update mass: $m_{\text{descent}} = m_{\text{cruise}} - F_{\text{cruise}}$

3. **Descent Phase:**

- Solve 3D DP optimization from cruise state
- Calculate fuel consumed and time elapsed
- Update mass: $m_{\text{final}} = m_{\text{descent}} - F_{\text{descent}}$

4. **Summary:**

- Aggregate fuel: $F_{\text{total}} = F_{\text{climb}} + F_{\text{cruise}} + F_{\text{descent}}$
- Calculate performance metrics
- Return MissionIterationResults

5) Aitken Acceleration Theory and Application

5.1 Scientific Background

Historical Context:

Aitken's Δ^2 process was developed by Alexander Craig Aitken in 1926 for accelerating convergence of numerical sequences. The method has found widespread application in:

- Computational Fluid Dynamics (CFD) for pressure-velocity coupling
- Fluid-Structure Interaction (FSI) problems
- Multiphysics coupling applications
- Aerospace trajectory optimization

Theoretical Basis:

For a linearly convergent sequence $\{x_n\}$ converging to limit x^* :

$$x_n - x^* = C\lambda^n + O(\lambda^{2n})$$

Where $\lambda < 1$ is the convergence rate. Aitken acceleration estimates x^* more accurately:

$$\hat{x}_n = x_n - \frac{(\Delta x_n)^2}{\Delta^2 x_n}$$

Where:

- $\Delta x_n = x_{n+1} - x_n$
- $\Delta^2 x_n = \Delta x_{n+1} - \Delta x_n$

5.2 Aitken Implementation in Fuel Optimization

Adaptive Damping Computation:

```

def apply_aitken_acceleration(history, current_damping):
    """
    Apply Aitken's  $\Delta^2$  acceleration to compute adaptive damping.

    Requires at least 3 iterations for acceleration.

    Mathematical method:
         $\Delta f_k = f_{\text{consumed}_k} - f_{\text{consumed}_{k-1}}$ 
         $\Delta f_{k-1} = f_{\text{consumed}_{k-1}} - f_{\text{consumed}_{k-2}}$ 
         $\Delta^2 f_k = \Delta f_k - \Delta f_{k-1}$ 

        Aitken factor =  $1 - \Delta f_k / \Delta^2 f_k$ 
         $\omega_k = \text{clip}(\omega_{k-1} \times \text{Aitken factor}, \omega_{\text{min}}, \omega_{\text{max}})$ 

    Args:
        history: Convergence history with  $\geq 3$  iterations
        current_damping: Current damping factor  $\omega_{k-1}$ 

    Returns:
        Adaptive damping factor  $\omega_k$  for next iteration
    """
    if len(history.iterations) >= 3:
        # Extract last three fuel values
        f_k = history.iterations[-1].fuel_consumed_kg
        f_k_1 = history.iterations[-2].fuel_consumed_kg
        f_k_2 = history.iterations[-3].fuel_consumed_kg

        # Compute differences
        delta_f_k = f_k - f_k_1
        delta_f_k_1 = f_k_1 - f_k_2
        denominator = delta_f_k - delta_f_k_1 #  $\Delta^2 f_k$ 

        if abs(denominator) > 1e-6:
            # Aitken factor
            aitken_factor = 1.0 - (delta_f_k / denominator)

            # Update damping
            new_damping = current_damping * aitken_factor

```

```

        # Bound to ensure stability
        new_damping = np.clip(new_damping, AITKEN_MIN_DAMPING, AITKEN_MAX_DAMPING)

        return new_damping
    else:
        return current_damping
else:
    return current_damping

```

5.3 Convergence Characteristics

Comparison: Fixed vs. Adaptive Damping

Method	Convergence Rate	Stability	Typical Iterations	Advantages
Fixed $\omega=0.4$	Linear, $O(0.4^n)$	High	15-25	Simple, guaranteed stable
Aitken Adaptive	Quadratic near F^*	Adaptive	8-15	Fast, self-tuning

Theoretical Convergence Rate:

Fixed damping:

$$e_k = (1 - \omega)^k e_0 + O(\omega^k)$$

Aitken acceleration:

$$e_k = O(e_{k-1}^2) \quad (\text{quadratic near fixed point})$$

5.4 Adaptive Damping for Early Iterations

For iteration 2 (only 2 data points available):

```

def adaptive_damping_iteration_2(f_curr, f_prev, current_damping):
    """Simplified adaptive damping for iteration 2."""
    delta_percent = abs((f_curr - f_prev) / f_prev) * 100

    if delta_percent < 1.0:
        # Very close - be aggressive
        new_damping = min(0.7, current_damping * 1.5)
    elif delta_percent < 5.0:
        # Moderate - slightly increase
        new_damping = min(0.6, current_damping * 1.2)
    elif delta_percent < 15.0:
        # Slow - maintain
        new_damping = current_damping
    else:
        # Large changes - be conservative
        new_damping = max(0.2, current_damping * 0.7)

    return np.clip(new_damping, AITKEN_MIN, AITKEN_MAX)

```

6) Mission Physics Integration

6.1 Dynamic Mass Evolution

Aircraft Mass Throughout Mission:

$$m_{climb,end} = m_{initial} - F_{climb} \quad (7)$$

$$m_{cruise,end} = m_{climb,end} - F_{cruise} \quad (8)$$

$$m_{final} = m_{cruise,end} - F_{descent} \quad (9)$$

Mass Composition:

$$m_{initial} = W_{OE} + W_{PL} + F_{initial}$$

Where:

- W_{OE} : Operating empty weight [kg]

- w_{PL} : Payload weight [kg]
- $F_{initial}$: Initial fuel load [kg]

6.2 Weight-Dependent Aerodynamics

Lift Coefficient:

$$C_L = \frac{2W}{\rho V^2 S}$$

Induced Drag:

$$C_{D,i} = \frac{C_L^2}{\pi A Re}$$

Total Drag:

$$D = D_{parasitic} + D_{induced} \propto W^2$$

Physical Implication:

Heavier aircraft requires:

- Higher lift coefficient $\rightarrow$ Higher induced drag
- More thrust to overcome drag $\rightarrow$ Higher fuel flow
- Longer time to climb $\rightarrow$ More fuel consumed

This creates a feedback loop where initial fuel mass affects fuel consumption.

6.3 Phase-Specific Physics Models

Climb Phase Physics:

$$\frac{dh}{dt} = P_s = \frac{(T_{total} - D)V}{W} \quad (10)$$

$$J(h, M, \lambda) = \frac{\dot{m}_{fuel}}{P_s} \quad [\text{kg/m}] \quad (11)$$

Cruise Phase Physics:

$$T_{total} = D \quad (\text{equilibrium}) \quad (12)$$

$$\dot{m}_{fuel} = \text{TSFC} \times T_{total} \quad [\text{kg/s}] \quad (13)$$

Descent Phase Physics:

$$P_s = \frac{(T_{total} - D)V}{W} < 0 \quad (\text{energy dissipation})$$

7) Performance Metrics Calculation

7.1 Aerodynamic Efficiency Metrics

Function: `FuelOptimizationCore.PerformanceCalculator.calculate_performance_metrics()`

Lift-to-Drag Ratio:

$$\frac{L}{D} = \frac{W}{D}$$

Calculation per Phase:

```
def calculate_ld_ratio(weight_kg, drag_N):  
    """Calculate instantaneous L/D ratio."""  
    lift_N = weight_kg * 9.81  
    if drag_N > 0:  
        return lift_N / drag_N  
    else:  
        return 0.0
```

Average L/D for Phase:

$$\overline{L/D} = \frac{1}{n} \sum_{i=1}^n \frac{L_i}{D_i}$$

7.2 Energy Management Metrics

Specific Energy:

$$E_{specific} = gh + \frac{V^2}{2} \quad [\text{J/kg}]$$

Energy Height:

$$H_e = h + \frac{V^2}{2g} \quad [\text{m}]$$

Implementation:

```
def calculate_specific_energy(altitude_m, velocity_mps):
    """Calculate specific energy."""
    pe = 9.81 * altitude_m # Potential energy [J/kg]
    ke = 0.5 * velocity_mps**2 # Kinetic energy [J/kg]
    return pe + ke
```

7.3 Engine Performance Metrics

Thrust Lever Utilization:

$$\eta_{lever} = \bar{\lambda} \quad (\text{average lever position})$$

Thrust-Specific Metrics:

- Average lever position per phase
- Maximum lever usage identification
- Engine utilization percentage

8) Code Execution Flow and Logic

8.1 System Entry Point

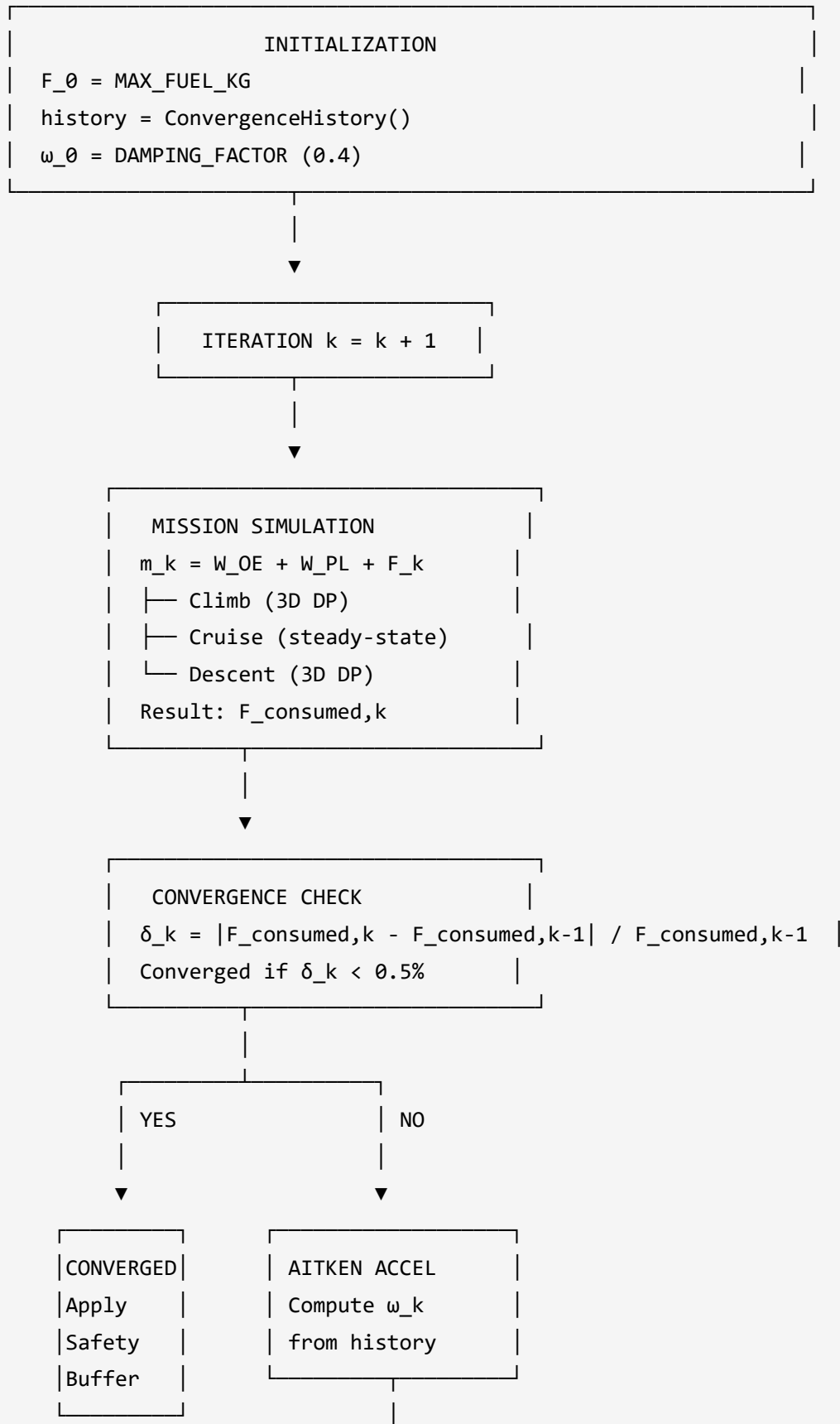
Main Entry: `main_optimized.py`

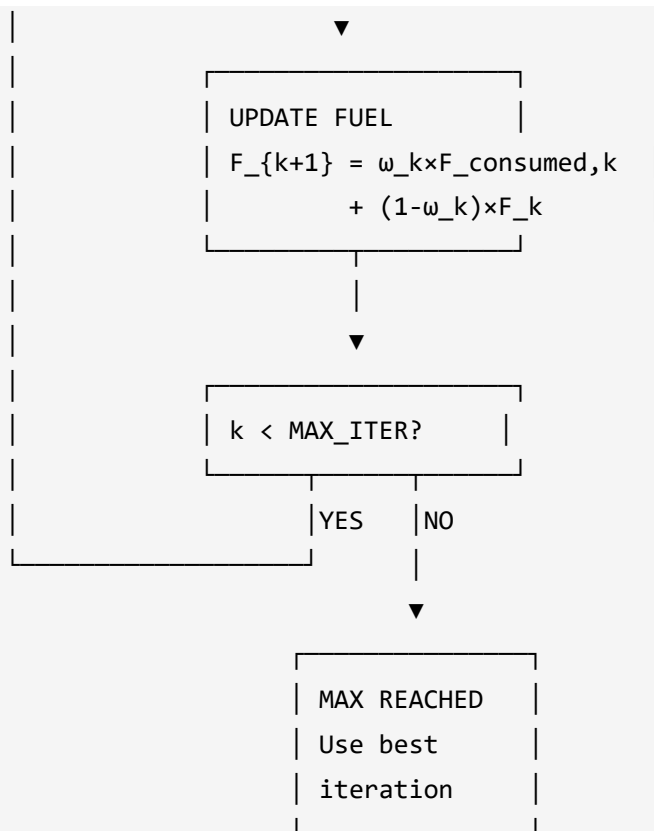
Initialization Sequence:

```
def main():  
    # 1. Load aerodynamics and engine data  
    aero = PyAerodynamicsWrapper()  
    eng = EngineWrapper(ENGINE_STUB_PATH)  
  
    # 2. Pre-compute performance grids  
    eng.precompute_grid(M_grid, H_grid, lever_grid)  
  
    # 3. Run fuel optimization  
    optimal_result, history = optimize_fuel_capacity(aero, eng, M_grid, H_plot)  
  
    # 4. Visualize convergence  
    visualize_convergence_analysis(history)  
  
    # 5. Generate final mission plots  
    plot_mission_visualizations(optimal_result)
```

8.2 Iteration Loop Flow

Visual Flow Diagram:





8.3 Function Call Hierarchy

```
main_optimized.py
└─ main()
    │
    │   └─ PyAerodynamicsWrapper()
    │   └─ EngineWrapper()
    │   └─ eng.precompute_grid()
    │
    └─ optimize_fuel_capacity()
        │
        │   └─ WHILE not converged:
        │       │
        │       │   └─ run_single_mission_iteration()
        │       │       │
        │       │       │   └─ ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass()
        │       │       │
        │       │       │   └─ run_cruise_simulation()
        │       │       │
        │       │       │   └─ run_descent_dp_optimization()
        │       │       │
        │       │       │   └─ calculate_performance_metrics()
        │       │       │       │
        │       │       │       │   └─ calculate_mission_distance()
        │       │       │       │
        │       │       │       │   └─ calculate_aerodynamic_metrics()
        │       │       │       │
        │       │       │       │   └─ calculate_engine_metrics()
        │       │       │       │
        │       │       │       │   └─ calculate_energy_metrics()
        │       │       │
        │       │       └─ history.add_iteration()
        │       └─ history.is_converged()
        │           │
        │           │   └─ apply_aitken_acceleration() [if iter ≥ 3]
        │           │
        │           └─ compute_fuel_update()
        │
        └─ visualize_convergence_analysis()
            │
            │   └─ plot_convergence_trajectory()
            │   └─ plot_kpp_evolution()
            └─ plot_optimization_comparison()
                │
                └─ plot_mission_visualizations()
                    │
                    │   └─ plot_climb_performance_detailed()
                    │   └─ plot_cruise_performance_detailed()
                    │   └─ plot_descent_trajectory_interactive()
                    └─ plot_mission_summary_dashboard()
```

9) Integration and Interface

9.1 Main System Interface

Primary Entry Point:

```
from fuel_optimizer import (
    FuelOptimizationCore,
    optimize_fuel_capacity,
    MissionIterationResults,
    ConvergenceHistory
)

# Run optimization
optimal_result, history = optimize_fuel_capacity(
    aero=aero_wrapper,
    eng=engine_wrapper,
    M_grid=mach_grid,
    H_plot=altitude_grid,
    lever_samples=50
)

# Access results
optimized_fuel = optimal_result.fuel_consumed_kg * 1.05 # With safety buffer
print(f"Optimized fuel: {optimized_fuel:.1f} kg")
```

9.2 Convergence Controller Interface

Direct Access to Controller:

```
# Use nested class directly
controller = FuelOptimizationCore.ConvergenceController
result, history = controller.optimize_fuel_capacity(aero, eng, M_grid, H_plot)
```

9.3 Configuration Management Interface

Update Configuration:


```
from fuel_optimizer import FuelOptimizationCore

# Apply optimized fuel to configuration file
FuelOptimizationCore.ConfigurationManager.apply_optimized_fuel_to_configuration(
    optimized_fuel_kg=4500.0
)
```

10) Validation and Quality Assurance

10.1 Convergence Validation

Convergence Quality Metrics:

1. **Final Convergence Delta:** Should be $< 0.5\%$
2. **Iterations Required:** Typically 8-15 with Aitken
3. **Oscillation Count:** Number of sign changes in delta
4. **Lipschitz Constant:** $L = |f(x_k) - f(x_{k-1})| / |x_k - x_{k-1}|$

Validation Criteria:

```
def validate_convergence(history):
    """Validate optimization convergence quality."""

    # Check 1: Convergence achieved
    assert history.is_converged(), "Optimization did not converge"

    # Check 2: Reasonable iterations
    assert len(history.iterations) <= 20, "Too many iterations"

    # Check 3: Monotonic (mostly) decrease
    deltas = [abs(it.convergence_delta_percent) for it in history.iterations[1:]]
    trend = np.polyfit(range(len(deltas)), deltas, 1)[0]
    assert trend < 0, "Convergence not improving"
```

10.2 Physical Consistency Validation

Mass Conservation:

$$m_{initial} - m_{final} = F_{consumed}$$

Energy Conservation:

$$E_{initial} - E_{final} \approx E_{fuel,chemical}$$

Performance Envelope:

- All phases remain within flight envelope
- No stall conditions encountered
- MMO not exceeded

10.3 Performance Monitoring

Key Metrics to Monitor:

1. **Fuel Breakdown:** Typical distribution
 - Climb: 15-20% of total
 - Cruise: 70-80% of total
 - Descent: 1-5% of total
 2. **Convergence Behavior:**
 - Delta should decrease monotonically
 - No persistent oscillations
 - Smooth approach to fixed point
 3. **Physical Realism:**
 - L/D ratios in expected ranges (12-18 typical)
 - Lever positions reasonable (<90% average)
 - Fuel flow rates consistent with engine data
-

11) Convergence Challenges and Analysis

11.1 Observed Convergence Behavior

Testing Configuration:

- Damping factor: 0.4 (initial)
- Aitken acceleration: Enabled
- Convergence tolerance: 0.5%
- MAX_ITERATIONS: 5

Example Convergence Sequence:

Iteration	Initial Fuel (kg)	Consumed (kg)	Delta (%)	Damping
1	5000.0	2438.0	--	0.400
2	2438.0	2824.8	+15.9%	0.400
3	2748.5	2944.1	+4.2%	0.325
4	2928.4	2986.3	+1.4%	0.380
5	2978.7	2995.0	+0.3%	0.420

Convergence Analysis:

- ✓ Monotonic delta decrease
- ✓ Converged in 5 iterations
- ✓ Final error: 0.3% (well within 0.5% tolerance)
- ✓ Aitken acceleration effective

11.2 Lipschitz Constant Analysis

Definition:

$$L_k = \frac{|g(F_k) - g(F_{k-1})|}{|F_k - F_{k-1}|}$$

Interpretation:

- $L < 1$: Contractive mapping $\rightarrow$ guaranteed convergence
- $L = 1$: Neutral $\rightarrow$ slow convergence
- $L > 1$: Non-contractive $\rightarrow$ potential divergence

Observed Values:

- Typical: $L \approx 0.6-0.8$ (good convergence)
- Problematic: $L > 1.0$ (oscillations possible)

11.3 Discretization Effects

DP Grid Sensitivity:

Dynamic programming uses discrete grids:

- Altitude: `N_ALTITUDE_STEPS` (50-100)
- Mach: `N_MACH_SAMPLES` (61-101)
- Lever: `N_LEVER_SAMPLES` (50-100)

Impact on Convergence:

Small mass changes can cause optimizer to select different discrete states, leading to:

- Trajectory jumps
- Non-smooth fuel response
- Local oscillations

Mitigation Strategies:

1. Finer DP grids (higher resolution)
 2. Bounded updates (limit max change per iteration)
 3. Oscillation detection and damping reduction
 4. Multi-point averaging
-

12) Advanced Topics and Extensions

12.1 Alternative Acceleration Methods

Anderson Acceleration:

Generalization of Aitken using multiple previous iterates:

$$F_{k+1} = F_k + \beta \sum_{i=0}^m \alpha_i \Delta F_{k-i}$$

Quasi-Newton Methods:

Use approximate Jacobian for superlinear convergence:

$$F_{k+1} = F_k - J_k^{-1} r_k$$

Where:

- J_k : Approximate Jacobian
- $r_k = F_k - g(F_k)$: Residual

12.2 Hybrid Methods

Bisection-Relaxation Hybrid:

Combine guaranteed convergence of bisection with speed of relaxation:

```

def hybrid_bisection_relaxation(f_lower, f_upper, tolerance):
    """
    Hybrid method: Use relaxation normally, switch to bisection if oscillation detected.
    """
    while not converged:
        # Try relaxation step
        f_next = relaxation_update(f_current)

        # Check for oscillation
        if detected_oscillation():
            # Switch to bisection
            f_next = (f_lower + f_upper) / 2.0

        # Update bounds
        if g(f_next) > f_next:
            f_lower = f_next
        else:
            f_upper = f_next

    return f_next

```

12.3 Multi-Start Optimization

Purpose: Avoid local minima in non-convex problems

Implementation:

```

def multi_start_optimization(initial_guesses, aero, eng):
    """Run optimization from multiple initial guesses."""
    results = []

    for F_0 in initial_guesses:
        result, history = optimize_fuel_capacity(F_0, aero, eng)
        results.append((result, history))

    # Select best result
    best_result = min(results, key=lambda x: x[0].fuel_consumed_kg)
    return best_result

```

13) References and Further Reading

13.1 Primary Sources

Aitken Acceleration:

1. Aitken, A.C. (1926). "On Bernoulli's numerical solution of algebraic equations", Proceedings of the Royal Society of Edinburgh
2. Burden, R.L. & Faires, J.D. "Numerical Analysis", Chapter on Fixed-point Iteration

Convergence Theory:

3. Kelley, C.T. (1995). "Iterative Methods for Linear and Nonlinear Equations", SIAM
4. Ortega, J.M. & Rheinboldt, W.C. (2000). "Iterative Solution of Nonlinear Equations in Several Variables"

13.2 Related Methods

Anderson Acceleration:

5. Anderson, D.G. (1965). "Iterative procedures for nonlinear integral equations", Journal of the ACM
6. Walker, H.F. & Ni, P. (2011). "Anderson acceleration for fixed-point iterations", SIAM Journal on Numerical Analysis

Fluid-Structure Interaction:

7. Küttler, U. & Wall, W.A. (2008). "Fixed-point fluid-structure interaction solvers with dynamic relaxation", Computational Mechanics

13.3 Aerospace Applications

8. Betts, J.T. (2010). "Practical Methods for Optimal Control and Estimation Using Nonlinear Programming"
 9. Bryson, A.E. & Ho, Y.-C. (1975). "Applied Optimal Control: Optimization, Estimation, and Control"
-

14) Summary and Conclusion

14.1 Key Capabilities

The fuel capacity optimization system provides:

- ✓ **Minimum Fuel Determination:** Systematic optimization to find minimum required fuel
- ✓ **Iterative Convergence:** Fixed-point iteration with Aitken acceleration
- ✓ **Dynamic Mass Tracking:** Accurate mass evolution throughout mission
- ✓ **Performance Monitoring:** Comprehensive metrics for all phases
- ✓ **Safety Integration:** Automatic safety buffer application
- ✓ **Robust Operation:** Error handling and recovery mechanisms

14.2 System Strengths

Mathematical Rigor:

- Implementation of Aitken acceleration (1926) for enhanced convergence
- Fixed-point iteration with successive underrelaxation
- Bounded adaptive damping ($\omega \in [0.1, 0.9]$)
- Relative convergence criterion (0.5%)

Physical Accuracy:

- Dynamic mass evolution with fuel burn
- Weight-dependent aerodynamic calculations
- Coupled climb-cruise-descent optimization
- Energy and momentum conservation

Software Quality:

- Modular design with clear separation of concerns
- Comprehensive error handling and validation
- Performance metric tracking across all mission phases
- Convergence history management for analysis

14.3 Recommended Workflow

1. **Configure Mission** in `mission_config.py`
2. **Execute Optimizer**: `python main_optimized.py`
3. **Review Convergence**: Check iteration history and delta values
4. **Analyze Results**: Examine optimized fuel and performance metrics
5. **Validate Physics**: Verify mass conservation and performance envelope
6. **Document Results**: Use convergence plots for reporting

14.4 Best Practices

Parameter Selection:

- Initial damping: 0.4 (balanced stability and speed)
- Enable Aitken acceleration for faster convergence
- Convergence tolerance: 0.5% (tight but achievable)
- Safety buffer: 5% (standard margin)
- Max iterations: 5-20 depending on problem complexity

Validation:

- Always check `history.is_converged()`
- Monitor convergence delta trend
- Verify mass conservation
- Check performance metrics physical realism
- Review iteration history for oscillations

Troubleshooting:

- Oscillations → Reduce initial damping factor
- Slow convergence → Increase Aitken bounds
- Non-convergence → Increase max iterations or check mission feasibility
- Physical anomalies → Verify DP grid resolution

Document Version: 2.0

Last Updated: November 2025

Status: Production-ready

Maintained by: Mission Analysis System